Lab 1: Lập trình Shell trên Linux (Shell Programming on Linux)

Hướng dẫn thực hành Lập trình hệ thống

Trường Đại học Công nghệ - ĐHQGHN

Ngày 19 tháng 9 năm 2026

Mục lục

1. Mở đầu: vì sao bắt đầu bằng shell
2. Shell và cấu trúc dòng lệnh
3. Hệ thống tệp và đường dẫn
4. Quyền truy cập file
5. Lệnh quản lý file và thư mục
6. Chuyển hướng luồng (redirection)
7. Pipe, IPC và grep
8. Shell script
9. Gỡ lỗi và tra cứu
10. Bài tập thực hành
11. Tổng kết

Nội dung

1. Mở đầu: vì sao bắt đầu bằng shell
2. Shell và cấu trúc dòng lệnh
3. Hệ thống tệp và đường dẫn
4. Quyền truy cập file
5. Lệnh quản lý file và thư mục
6. Chuyển hướng luồng (redirection)
7. Pipe, IPC và grep
8. Shell script
9. Gỡ lỗi và tra cứu
10. Bài tập thực hành
11. Tổng kết

Mục tiêu buổi thực hành

Sau buổi hôm nay, sinh viên phải làm được

1. **Định vị và thao tác:** di chuyển trong cây thư mục, tạo/sao chép/xoá/đổi tên file, đọc và sửa quyền truy cập.
2. **Quan sát hệ thống:** liệt kê tiến trình, chuyển hướng luồng vào/ra/lỗi, ghép lệnh bằng pipe.
3. **Tự động hoá:** viết shell script có tham số, biến, điều kiện, vòng lặp, hàm.
4. **Gỡ lỗi và tra cứu:** dùng `bash -x`, `set -euo pipefail`, `man`.

Sản phẩm cần nộp

8 bài tập: 3 bài dòng lệnh, 1 bài redirection/pipe, 4 script (`greeting.sh`, `test.sh`, `dec2hex.sh`, `fibonacci.sh`).

Shell là cửa sổ quan sát được vào kernel

Bài giảng tuần 1: shell là **một tiến trình người dùng bình thường**, không có đặc quyền. Mọi việc nó làm đều đi qua system call.

Việc trong shell	System call bên dưới	Sẽ viết bằng C ở
`ls`, `cat file`	`open`, `read`, `write`, `close`	File I/O
`chmod u+x f`	`chmod`, `stat`	Quyền truy cập
Chạy một lệnh	`fork` + `execve` + `wait`	Quản lý tiến trình
`cmd > file`	`open` + `dup2`	File descriptor
`cmd1 | cmd2`	`pipe` + `fork` + `dup2`	IPC
`kill -9 PID`	`kill`	Signal

Thông điệp chính

Hiểu ở cấp shell trước thì phần C sau này gần như không còn khái niệm mới, chỉ còn cú pháp.

Nội dung

1. Mở đầu: vì sao bắt đầu bằng shell
2. Shell và cấu trúc dòng lệnh
3. Hệ thống tệp và đường dẫn
4. Quyền truy cập file
5. Lệnh quản lý file và thư mục
6. Chuyển hướng luồng (redirection)
7. Pipe, IPC và grep
8. Shell script
9. Gỡ lỗi và tra cứu
10. Bài tập thực hành
11. Tổng kết

Cấu trúc một dòng lệnh

`command [options] [arguments]`

- `command` – tên lệnh cần thực thi (`ls`, `mkdir`, `echo`).
- `options` – *tuỳ chọn*, thay đổi hành vi; bắt đầu bằng `-` (ngắn) hoặc `--` (dài).
- `arguments` – *đối số*, đối tượng mà lệnh tác động lên.

Ví dụ: `ls -l /home/user`

- `ls` → lệnh (liệt kê nội dung thư mục)
- `-l` → tuỳ chọn (long format: quyền, chủ sở hữu, kích thước)
- `/home/user` → đối số (thư mục cần liệt kê)

Gộp các tùy chọn ngắn

Gộp tuỳ chọn ngắn: `ls -lah`. Tuỳ chọn dài cần `=` hoặc khoảng trắng: `--color=auto`. Dấu `--` báo "hết tuỳ chọn": `rm -- -tenfile-la`.

Shell phân tích dòng lệnh theo trình tự cố định

1. Tách token theo khoảng trắng, toán tử
2. Brace expansion `{1,2}`, `{1..5}`
3. Tilde expansion `~` → `$HOME`
4. Thế biến `$var`, `$1`
5. Thế lệnh `$(command)`
6. Thế số học `$(( expr ))`
7. **Word splitting** theo `IFS`
8. Glob `*`, `?`, `[abc]`
9. Redirection, rồi mới thực thi

Bước 7 - Nguyên nhân nhiều lỗi script

Nếu `f="Bao cao.txt"` thì `rm $f` trở thành `rm Bao cao.txt` – hai đối số, xoá sai file. **Cách chữa duy nhất: luôn bọc biến trong nháy kép** `rm "$f"`.

Lệnh built-in và lệnh ngoài

- **Built-in:** biên dịch bên trong shell (`cd`, `echo`, `read`, `export`, `test`, `[`).
- **Lệnh ngoài:** file thực thi trên đĩa, tìm theo `$PATH` (`ls` là `/bin/ls`).

```
type cd         # -> cd is a shell builtin
type ls         # -> ls is /usr/bin/ls
type -a echo    # liet ke MOI phien ban tim thay
```

Câu hỏi vấn đáp: vì sao `cd` buộc phải là built-in?

Thư mục làm việc là thuộc tính *của tiến trình*. Nếu `cd` là chương trình ngoài, shell sẽ `fork` ra con, con gọi `chdir()` rồi *chết* – thư mục của shell cha không đổi.

Nội dung

1. Mở đầu: vì sao bắt đầu bằng shell
2. Shell và cấu trúc dòng lệnh
3. Hệ thống tệp và đường dẫn
4. Quyền truy cập file
5. Lệnh quản lý file và thư mục
6. Chuyển hướng luồng (redirection)
7. Pipe, IPC và grep
8. Shell script
9. Gỡ lỗi và tra cứu
10. Bài tập thực hành
11. Tổng kết

"Mọi thứ đều là file" – nhưng có nhiều loại file

Ký tự *đầu tiên* trong output `ls -l` cho biết loại file:

Ký tự	Loại	Ý nghĩa
`-`	Regular file	Dãy byte: văn bản, ảnh, file thực thi
`d`	Directory	File *liệt kê các file khác* (tên → inode)
`l`	Symbolic link	File chứa đường dẫn tới file khác
`c` / `b`	Character / Block device	`/dev/tty` / `/dev/sda1`
`p` / `s`	Named pipe / Socket	Kênh IPC

Chú ý: Directory là file liệt kê file khác

Nó giải thích: mỗi thư mục luôn có `.` và `..`; quyền `w` trên thư mục cho phép *xoá file bên trong*; `ls -al` trên thư mục rỗng vẫn hiện 2 entry.

Các thư mục chuẩn (FHS)

- `/` – gốc của toàn bộ cây
- `/home` – thư mục nhà người dùng
- `/bin`, `/usr/bin` – file thực thi
- `/etc` – file cấu hình hệ thống
- `/var` – log, cache, dữ liệu biến động
- `/tmp` – file tạm, xoá khi reboot
- `/dev` – file thiết bị
- `/proc` – hệ thống file *ảo* do kernel sinh
- `/lib` – thư viện dùng chung

Thử ngay – minh hoạ đẹp nhất cho triết lý "mọi thứ là file"

```
cat /proc/cpuinfo   cat /proc/$$/status   ls /proc/$$/
```

Lệnh `ps` thực chất chỉ đọc và định dạng lại nội dung `/proc`.

Đường dẫn tuyệt đối và tương đối

- **Tuyệt đối:** tính từ gốc, luôn bắt đầu bằng `/`, luôn chỉ tới đúng một chỗ bất kể bạn đứng ở đâu. Ví dụ `/home/jono`.
- **Tương đối:** tính từ thư mục hiện tại, không bắt đầu bằng `/`. Ví dụ `./work`.

Ký hiệu	Nghĩa	Ví dụ
`.`	Thư mục hiện tại	`./script.sh`
`..`	Thư mục cha	`cd ..`
`~`	Thư mục nhà của tôi	`cd ~/Labs`
`-`	Thư mục *vừa rời khỏi*	`cd -`

Lưu ý Excercise 2

`cd` *không có đối số* không phải "ở nguyên chỗ cũ" – nó tương đương `cd $HOME`. Lệnh đầu tiên của Bài 2 đã đưa bạn về thư mục nhà, nên các bước sau **không** còn ở `Lab1`.

Nội dung

1. Mở đầu: vì sao bắt đầu bằng shell
2. Shell và cấu trúc dòng lệnh
3. Hệ thống tệp và đường dẫn
4. Quyền truy cập file
5. Lệnh quản lý file và thư mục
6. Chuyển hướng luồng (redirection)
7. Pipe, IPC và grep
8. Shell script
9. Gỡ lỗi và tra cứu
10. Bài tập thực hành
11. Tổng kết

Ba nhóm người dùng, ba quyền - Tam quyền phân lập

Nhóm đối tượng

- u – user/owner: người tạo file
- g – group: cùng nhóm (`/etc/group`)
- o – others: mọi người còn lại
- a – all $= u + g + o$

Quyền

KH	Số	Ý nghĩa
r	4	Read – đọc nội dung
w	2	Write – sửa/xoá
x	1	Execute – chạy được

Giải thích output `ls -l myfile.txt`

```
-rw-r-r-  1 user group 1234 Feb 15 12:00 myfile.txt
```

-	rw-	r-	r-
loại file	owner: đọc+ghi	group: chỉ đọc	others: chỉ đọc

chmod: hai cách biểu diễn

```
chmod u+x   script.sh        # them quyen chay cho chu so huu
chmod o+w   Exercise1.txt    # them quyen ghi cho others  <- Bai tap 1.8
chmod go-w  file.txt         # bo quyen ghi cua group va others
chmod u=rw,g=r,o= file.txt   # DAT LAI tuyet doi
chmod 755   script.sh        # rwxr-xr-x  (4+2+1, 4+1, 4+1)
```

Octal	Ký hiệu	Dùng cho	Ghi chú
644	rw-r--r-	File văn bản	Mặc định nhiều editor
755	rwxr-xr-x	Script, thư mục	Ai cũng chạy/vào được
600	rw----	Khoá SSH, mật khẩu	Chỉ chủ sở hữu
777	rwxrwxrwx	*Hầu như không bao giờ*	Xem cảnh báo

chmod 777 là cấp quyền rất đần cho mọi vấn đề về quyền

Nó cho bất kỳ ai sửa hoặc thay thế script mà bạn sẽ chạy. Sử dụng 777 là dấu hiệu của việc đoán thay vì hiểu.

`rwx` trên thư mục có ý nghĩa hoàn toàn khác trên file

Quyền	Trên regular file	Trên directory
`r`	Đọc nội dung file	Liệt kê *tên* các entry (`ls`)
`w`	Sửa/ghi nội dung file	**Tạo, xoá, đổi tên** entry bên trong
`x`	Thực thi như chương trình	**Đi vào** (`cd`) và truy cập file theo tên

Lưu ý mở rộng

1. **Xoá được file không phụ thuộc quyền trên file**, mà phụ thuộc quyền `w` trên *thư mục chứa nó*. Bạn xoá được file `r--r--r--` của người khác nếu có `w` trên thư mục.
2. Thư mục có `r` nhưng không `x`: thấy tên mà không đọc được gì. Có `x` mà không `r`: không liệt kê được, nhưng *biết đúng tên* thì vẫn mở được.

Bổ sung: `umask` và sticky bit

File mới = 666 − `umask` (thường 022 → 644); thư mục mới = 777 − 022 = 755. `/tmp` có `drwxrwxrwt`: chữ `t` là sticky bit – ai cũng ghi được nhưng chỉ chủ sở hữu mới xoá được file của mình.

Nội dung

1. Mở đầu: vì sao bắt đầu bằng shell
2. Shell và cấu trúc dòng lệnh
3. Hệ thống tệp và đường dẫn
4. Quyền truy cập file
5. Lệnh quản lý file và thư mục
6. Chuyển hướng luồng (redirection)
7. Pipe, IPC và grep
8. Shell script
9. Gỡ lỗi và tra cứu
10. Bài tập thực hành
11. Tổng kết

Bảng tra cứu (phần 1)

Lệnh	Cú pháp	Mô tả
`pwd`	`pwd`	In đường dẫn thư mục hiện tại
`cd`	`cd [dir]`	Chuyển thư mục
	`cd .. / cd ~ / cd -`	Lên cha / về nhà / về chỗ vừa rời
`ls`	`ls -l -a -h -d -R -t`	Chi tiết / file ẩn / dễ đọc / chính thư mục
`mkdir`	`mkdir [dir]`	Tạo thư mục
	`mkdir -p a/b/c`	Tạo cả chuỗi cha–con
`rmdir`	`rmdir [dir]`	Xoá thư mục *rỗng* (an toàn)
`touch`	`touch [file]`	Tạo file rỗng
`cp`	`cp [src] [dst]`	Sao chép file
	`cp -r [src] [dst]`	Sao chép thư mục (đệ quy)
`mv`	`mv [src] [dst]`	Di chuyển *hoặc* đổi tên
`rm`	`rm [file] / rm -r [dir]`	Xoá file / xoá cả cây

Lưu ý

`touch` không có trong bảng của tài liệu Lab, nhưng Bài tập 1 cần nó để "tạo một file".

Bảng tra cứu (phần 2)

Lệnh	Cú pháp	Mô tả
`chmod`	`chmod MODE FILE`	Sửa quyền truy cập
`cat`	`cat [file]`	In nội dung file
	`cat > [file]`	Tạo file mới, gõ nội dung, Ctrl+D để lưu
	`cat » [file]`	Ghi thêm vào cuối file
`echo`	`echo "text"`	In văn bản ra terminal
	`echo "t"> f`	Ghi văn bản vào file (*ghi đè*)
	`echo $VAR`	In giá trị biến môi trường
`head` / `tail`	`head -n 5 f`	5 dòng đầu / cuối; `tail -f` theo dõi log
`wc`	`wc -l -w -c f`	Đếm dòng / từ / byte
`find`	`find DIR -name "*.txt"`	Tìm theo tên, loại, kích thước
`sed`	`sed -n '3p' f`	In dòng thứ 3 (cần cho Bài tập 4)

Dễ toi - cần lưu ý

Đọc kỹ trước khi gõ

1. **Không có thùng rác.** `rm` gọi `unlink()` – không Undo, không Recycle Bin. Trước mỗi `rm -r`, chạy `ls` với cùng đối số để xem chính xác cái gì sẽ mất.
2. `rm -rf ~ /` (một khoảng trắng lạc chỗ) xoá cả thư mục nhà. Không gõ `-rf` theo quán tính.
3. `mv` **và** `cp` **ghi đè im lặng**, không hỏi gì. Dùng `-i` với dữ liệu thật.
4. **> khác ».** Một dấu > đặt sai chỗ ở Ex3 và Ex4 sẽ xoá sạch kết quả các bước trước – *lỗi mất điểm phổ biến nhất của Lab 1.*

Cách tự bảo vệ

`set -o noclobber` khiến > báo lỗi thay vì ghi đè file đã tồn tại; muốn ghi đè có ý thức thì dùng `>|`.

Nội dung

1. Mở đầu: vì sao bắt đầu bằng shell
2. Shell và cấu trúc dòng lệnh
3. Hệ thống tệp và đường dẫn
4. Quyền truy cập file
5. Lệnh quản lý file và thư mục
6. Chuyển hướng luồng (redirection)
7. Pipe, IPC và grep
8. Shell script
9. Gỡ lỗi và tra cứu
10. Bài tập thực hành
11. Tổng kết

Ba luồng chuẩn và file descriptor

File descriptor chỉ là một *số nguyên*: chỉ mục vào bảng file đang mở của tiến trình.

fd	Tên	Mặc định nối tới	Vai trò
0	`stdin`	bàn phím	Luồng dữ liệu vào
1	`stdout`	màn hình	Kết quả bình thường của lệnh
2	`stderr`	màn hình	Thông báo lỗi và cảnh báo

stdout và stderr trông giống nhau nhưng độc lập hoàn toàn

Nhờ tách riêng, bạn lưu được kết quả vào file mà vẫn thấy lỗi trên màn hình. Thói quen chuẩn trong script: **dữ liệu ra stdout, thông báo cho người đọc ra stderr** (`echo "Loi: khong tim thay file">&2`).

Bảng các toán tử chuyển hướng

Toán tử	Loại	Mô tả / ví dụ
>	Output	stdout vào file, **ghi đè**: `ls > files.txt`
»	Append output	stdout vào *cuối* file: `echo "x"» log.txt`
<	Input	stdin từ file: `sort < names.txt`
2>	Error	stderr vào file: `ls nodir 2> error.log`
2»	Append error	stderr vào cuối file
&>	Cả hai (bash)	`cmd &> output.txt`
2>&1	Gộp luồng	stderr về *nơi stdout đang trỏ tới*
>&2	Gộp luồng	stdout của lệnh đó sang stderr
/dev/null	Hố đen	Bỏ mọi thứ ghi vào: `cmd 2> /dev/null`
«, «<	Here-doc, here-string	Cấp nhiều dòng / một dòng làm stdin

Thứ tự của 2>&1 có ý nghĩa quyết định

```
cmd > out.txt 2>&1      # DUNG: ca stdout va stderr vao out.txt
cmd 2>&1 > out.txt      # SAI: stdout vao file, stderr van ra man hinh
```

Đọc từ trái sang phải – đây chính là hành vi của dup2()

- Trường hợp 1: fd 1 trỏ tới `out.txt` → fd 2 trỏ tới *cùng chỗ với fd 1* = `out.txt`.
- Trường hợp 2: fd 2 trỏ tới nơi fd 1 *đang* trỏ (màn hình) → fd 1 mới chuyển sang file.

`2>&1` **sao chép đích hiện tại** của fd 1, không tạo ràng buộc "mãi mãi theo fd 1".

Here-document và tee

```
cat > Hello.txt << 'EOF'
Hello everyone!
This is Advanced Programming Techniques course.
EOF
```

Nháy đơn quanh 'EOF' *tắt* việc thế biến trong khối – dùng khi nội dung có ký tự $.

```
ls -l | tee listing.txt          # vua ghi file vua in ra man hinh
ps -e | tee -a Ex4_results.txt   # -a = append
```

Ứng dụng ngay ở Bài tập 3

Here-doc ghi cả 5 dòng trong *một* lần – không thể sai thứ tự > và ».

Nội dung

1. Mở đầu: vì sao bắt đầu bằng shell
2. Shell và cấu trúc dòng lệnh
3. Hệ thống tệp và đường dẫn
4. Quyền truy cập file
5. Lệnh quản lý file và thư mục
6. Chuyển hướng luồng (redirection)
7. Pipe, IPC và grep
8. Shell script
9. Gỡ lỗi và tra cứu
10. Bài tập thực hành
11. Tổng kết

Pipe: cơ chế IPC đơn giản nhất

Mỗi tiến trình có không gian địa chỉ riêng ⇒ không đọc được biến của nhau ⇒ phải qua IPC (pipe, signal, shared memory, message queue, semaphore, socket).

```
ls | grep ".txt"      # noi stdout cua ls vao stdin cua grep
```

Ba điều hầu như không được nói trong tài liệu nhập môn

1. **Các lệnh trong pipeline chạy *đồng thời***, không tuần tự. Shell tạo tất cả tiến trình cùng lúc, nối bằng buffer trong kernel (64 KB). Nhờ vậy `grep` trên file 10 GB không cần 10 GB RAM.
2. **Pipe chỉ chuyển stdout**, không chuyển stderr. Muốn lọc cả lỗi: `cmd 2>&1 | grep ...`
3. **Mã trả về của pipeline là của lệnh *cuối cùng*.** `sai_lenh | wc -l` vẫn trả 0. Dùng `${PIPESTATUS[@]}` hoặc `set -o pipefail`.

grep: các tuỳ chọn hay dùng

```
grep [OPTIONS] PATTERN [FILE...]
```

Tuỳ chọn	Mô tả	Ví dụ
`-i`	Không phân biệt hoa/thường	`grep -i "error"f`
`-n`	Hiện số dòng	`grep -n "error"f`
`-v`	Hiện các dòng *không* khớp	`grep -v "error"f`
`-w`	Chỉ khớp *trọn từ*	`grep -w "error"f`
`-c`	Đếm *số dòng* khớp	`grep -c "error"f`
`-o`	In từng *lần khớp* riêng dòng	`grep -o "err"f | wc -l`
`-r`	Tìm đệ quy trong thư mục	`grep -r "error"/var/logs/`
`-E` / `-F`	Regex mở rộng / chuỗi thường	`grep -F "a.b"f`

`-c` đếm DÒNG, không đếm SỐ LẦN xuất hiện

Một dòng chứa "course" hai lần, `grep -c` vẫn trả về 1. Bài tập 4.1 yêu cầu đếm *số lần*, nên đáp án chặt chẽ là `grep -o | wc -l`.

Lưu ý: `ls | grep ".txt"` KHÔNG tìm file .txt

Trong regex, dấu `.` khớp **một ký tự bất kỳ**. Lệnh này khớp cả `myXtxt`, `atxtb`, `data.txt.bak`.

```
ls *.txt                                # dung glob cua shell -- don gian va dung
    nhat
ls | grep '\.txt$'                      # escape dau . va neo cuoi dong bang $
find . -maxdepth 1 -name '*.txt'        # ben vung voi ten file co khoang trang
```

Anti-pattern: parse output của `ls`

Tên file chứa khoảng trắng hoặc ký tự xuống dòng sẽ phá vỡ mọi thứ. Trong script hãy dùng glob, hoặc `find -print0 | xargs -0`.

Bộ công cụ lọc thường dùng cùng pipe

Lệnh	Công dụng
`wc`	`-l` dòng, `-w` từ, `-c` byte, `-m` ký tự
`sort`	`-n` theo số, `-r` ngược, `-k2` theo cột 2, `-u` loại trùng
`uniq`	Loại dòng trùng *liền kề* (nên `sort` trước); `-c` đếm số lần
`cut`	Lấy cột: `cut -d: -f1 /etc/passwd`
`tr`	Thay/xoá ký tự: `tr ' ' '\n'`
`sed`	Sửa dòng theo mẫu: `sed -n '3p'`, `sed 's/cu/moi/g'`
`awk`	Xử lý theo cột có logic: `awk '{s+=$1} END{print s}'`
`xargs`	Biến stdin thành đối số: `find . -name '*.o' | xargs rm`

Triết lý Unix

Mỗi công cụ làm *một* việc và làm tốt; sức mạnh đến từ việc **ghép** chúng lại. Shell mạnh ở việc ghép các chương trình, không mạnh ở việc *là* chương trình.

Nội dung

1. Mở đầu: vì sao bắt đầu bằng shell
2. Shell và cấu trúc dòng lệnh
3. Hệ thống tệp và đường dẫn
4. Quyền truy cập file
5. Lệnh quản lý file và thư mục
6. Chuyển hướng luồng (redirection)
7. Pipe, IPC và grep
8. Shell script
9. Gỡ lỗi và tra cứu
10. Bài tập thực hành
11. Tổng kết

Bốn bước tạo và chạy một script

1. **Tạo file** bằng editor: `vim script.sh`, `nano script.sh`
2. **Viết lệnh**, dòng đầu là shebang `#!/bin/bash`; lưu và thoát
3. **Cấp quyền thực thi**: `chmod +x script.sh`
4. **Chạy**: `./script.sh`

Shebang (#!) không phải chú thích

Hai byte đầu được *kernel* đọc khi `execve()` được gọi: kernel lấy đường dẫn phía sau làm trình thông dịch. Thiếu shebang, hành vi phụ thuộc môi trường – nguồn lỗi kinh điển.

Vì sao phải có `./` ở trước tên script?

Vì thư mục hiện tại *không* nằm trong `$PATH` – và không nên nằm trong đó, vì lý do an ninh: kẻ khác đặt file tên `ls` trong thư mục dùng chung là bạn chạy mã của họ.

Bốn cách chạy script và khác biệt giữa chúng

Cách chạy	**Cần `chmod +x`?**	**Chạy trong tiến trình nào**
`./script.sh`	Có	Tiến trình con; dùng trình thông dịch ở shebang
`bash script.sh`	Không	Tiến trình con; *bỏ qua* shebang, luôn dùng bash
`sh script.sh`	Không	Tiến trình con dùng `dash` – cú pháp bash sẽ lỗi
`source script.sh`	Không	**Chính shell hiện tại**

Hệ quả thực tế

Script chứa `cd /tmp` chạy bằng `./script.sh` *không* làm shell của bạn chuyển thư mục – `cd` xảy ra trong tiến trình con rồi con chết. Muốn script đổi môi trường của shell gọi nó thì buộc phải `source`. Đó cũng là lý do `~/.bashrc` được `source`, không phải thực thi.

vim: tối thiểu cần biết

Chế độ	Vào bằng	Dùng để
Normal (mặc định)	`Esc`	`h j k l` di chuyển; `yy` copy dòng, `dd` xoá dòng, `p` dán, `u` undo, `Ctrl+r` redo
Insert	`i` / `a` / `o`	Nhập và sửa văn bản
Command	`:`	`:w` lưu, `:q` thoát, `:wq` lưu và thoát, `:q!` thoát không lưu, `:!ls` chạy lệnh shell

Lời khuyên cho buổi hôm nay

Nếu mới học, dùng `nano` (Ctrl+O lưu, Ctrl+X thoát) để không mất thời gian vào editor. Nhưng học `vim` dần, vì trên server không GUI thường đó là lựa chọn duy nhất. `vimtutor` là khoá học 30 phút tốt nhất cho việc này.

Biến trong shell

```
salutation=Hello
echo $salutation          # -> Hello
salutation="Yes Dear"     # co khoang trang => PHAI dung nhay
salutation=7+5
echo $salutation          # -> 7+5   (shell KHONG tu tinh toan)
read salutation           # doc mot dong tu ban phim vao bien
```

Ba quy tắc cú pháp bắt buộc

1. **Không có khoảng trắng quanh dấu =**. `x = 5` bị hiểu là "chạy lệnh `x` với đối số = và 5".
2. Chuỗi chứa khoảng trắng phải đặt trong nháy.
3. Tên biến **phân biệt hoa/thường**: `$x` và `$X` là hai biến khác nhau.

Quoting – ba loại nháy

Dạng	Hành vi
`"$var"` nháy kép	Thế biến và thế lệnh *có* hiệu lực; word splitting và glob *bị vô hiệu*. **Dạng nên dùng mặc định.**
`'$var'` nháy đơn	Mọi thứ là ký tự thường. Dùng cho regex, mẫu glob.
`$var` không nháy	Thế biến rồi *tách từ* và *glob*. Chỉ dùng khi *cố ý* muốn tách.

```
f="Bao cao thang 9.txt"
rm  $f          # SAI: thanh 4 doi so
rm "$f"         # DUNG
echo '$HOME'  # -> $HOME      (nguyen van)
echo "$HOME"  # -> /home/user
```

Đây là nội dung tài liệu Lab không có mục nào, nhưng gây phần lớn lỗi script

Nếu chỉ nhớ một điều từ slide này: **bọc nháy kép quanh mọi biến, mọi lúc**.

Biến môi trường và biến tham số

Biến môi trường viết **HOA** theo quy ước, để phân biệt với biến người dùng tự đặt (viết thường).

Môi trường	Ý nghĩa
`$HOME`	Thư mục nhà
`$USER`	Tên người dùng
`$PWD`	Thư mục hiện tại
`$PATH`	Nơi dò tìm lệnh
`$TERM`	Loại terminal

Tham số	Ý nghĩa
`$0`	Tên của script
`$1`, `$2`	Tham số thứ 1, 2
`$#`	*Số lượng* tham số
`$@`	Mọi tham số, giữ phần tử
`$*`	Mọi tham số, ghép chuỗi
`$?`	Mã trả về lệnh vừa chạy
`$$`	PID của script

Lưu ý

Tham số thứ 10 trở đi **phải** có ngoặc nhọn: `${10}`, không phải `$10`.

"$@" và "$*" không đồng nghĩa

Với lời gọi `./s.sh "a b"c`:

Viết	Kết quả
`"$@"`	Hai từ: `"a b"` và `"c"` – **đúng**
`"$*"`	Một từ duy nhất: `"a b c"`
`$@` hoặc `$*` (không nháy)	Ba từ: `a`, `b`, `c` – mất cấu trúc

Quy tắc

Khi *chuyển tiếp* tham số cho lệnh khác: *luôn* dùng `"$@"` kèm nháy kép. Dùng `"$*"` chỉ khi muốn in một chuỗi gộp cho người đọc.

Lưu ý tài liệu Lab

Bảng biến tham số chỉ nêu `$@`, còn ví dụ lại dùng `$*`, mà không giải thích chúng khác nhau ⇒ lỗi im lặng khi tham số có khoảng trắng.

Điều kiện: test, [ ] và [[ ]]

Trên phần lớn hệ thống, [*chính là một lệnh*; khi dùng [thì đóng bằng] cho dễ đọc.

```
if test -f myfile.txt          if [ -f myfile.txt ]
then                           then
    ...                            ...
fi                             fi
```

Phải có khoảng trắng ngay sau [và ngay trước]

Vì [là *tên lệnh*: `[-f file]` bị hiểu là lệnh tên `[-f` → "command not found". Thiếu khoảng trắng trước] → "missing]".

Dạng mở rộng của bash

`[[ -f "$f" && "$f" == *.txt ]]` – không cần bọc nháy, hỗ trợ `&&`, so khớp mẫu và regex `=~`. `(( n > 5 ))` cho so sánh số với toán tử quen thuộc.

Ba bảng điều kiện

So sánh chuỗi

`s1 = s2`	bằng
`s1 != s2`	khác
`-n s`	không rỗng
`-z s`	rỗng

So sánh số

`-eq`	bằng
`-ne`	khác
`-gt`	lớn hơn
`-ge`	lớn hơn/bằng
`-lt`	nhỏ hơn
`-le`	nhỏ hơn/bằng
`!`	phủ định

Kiểm tra file

`-e f`	tồn tại
`-f f`	là regular file
`-d f`	là thư mục
`-r/-w/-x`	có quyền r/w/x
`-s f`	tồn tại, khác rỗng
`-L f`	là symlink

Bốn lỗi điều kiện tốn thời gian nhất

(1) Dùng `-eq` cho chuỗi hoặc `=` cho số: `[ 10 = 010 ]` *sai* nhưng `[ 10 -eq 010 ]` *đúng*. (2) Không bọc nháy: nếu `$name` rỗng thì `[ $name = "x"]` $\rightarrow$ lỗi cú pháp. (3) Dùng `==` trong `[ ]`: bash chấp nhận, POSIX `sh` thì không. (4) Nhầm `-e` (tồn tại) với `-f` (là regular file).

Biểu thức số học

```
x=0
x=$(( x + 1 ))       # CACH NEN DUNG: nhanh, khong tao tien trinh con
x=$(expr $x + 1)     # cach cu: goi lenh ngoai /usr/bin/expr
```

Lưu ý về mục 3.3.3

1. `X=0` rồi `X=$(( $x + 1))` – *gán* `X` nhưng *đọc* `$x`. Shell phân biệt hoa/thường nên `$x` rỗng; kết quả là 1 một cách tình cờ, và mãi là 1 nếu lặp lại.
2. `X=$(expr $x +1)` – **báo lỗi cú pháp**. `expr` là lệnh ngoài, nhận từng đối số riêng; `+1` là *một* đối số, không phải toán tử và toán hạng. Phải viết `expr $x + 1`. Cùng lý do, nhân phải escape: `expr 3 \* 4`.

Shell chỉ tính số nguyên

`echo $(( 7 / 2 ))` cho **3**, không phải 3.5. Muốn số thực: `echo "scale=3; 7/2kern.03em bc`. Đúng cái bẫy của tuần 2 với C (`9/5 = 1`, phải viết `9.0/5.0`).

Thế lệnh: phân biệt ba cú pháp giống nhau

```
now=$(date +%H:%M:%S)
nfiles=$(ls -1 | wc -l)
echo "Luc $now, co $nfiles file"
```

Cú pháp	Dùng để
`$(( ... ))`	*Thế số học*: tính biểu thức
`$( ... )`	*Thế lệnh*: chạy lệnh, lấy **output dạng chuỗi**
`${ ... }`	*Thế biến*: `${name}`, `${name:-default}`, `${#name}`

Lưu ý

`$(cmd)` cho *output*, **không** phải mã trả về. Muốn mã trả về thì đọc `$?` ngay sau lệnh. Cú pháp cũ ‘`cmd`‘ tương đương nhưng không lồng nhau được – dùng `$()`.

Cấu trúc điều khiển: if và for

```
if [ -f "$1" ]; then
    echo "$1 la file thuong"
elif [ -d "$1" ]; then
    echo "$1 la thu muc"
else
    echo "$1 khong ton tai" >&2
    exit 1
fi
```

```
for var in bar fud 43; do
    echo -n "$var "
done
for var in {1..5}; do ...; done
for i in $(seq 1 5); do ...; done
for (( i=0; i<5; i++ )); do ...; done
for f in *.txt; do ...; done
for arg in "$@"; do ...; done
```

Đính chính: {1..5} không chạy với #!/bin/sh

Brace expansion là tính năng *bash*. Trên Ubuntu /bin/sh là dash → vòng lặp chạy *một* lần với chuỗi {1..5}. Hoặc đổi shebang thành #!/bin/bash, hoặc dùng seq.

for f in $(ls) là mẫu sai

Tên file có khoảng trắng → mỗi mảnh thành một vòng lặp. Dùng glob trực tiếp for f in *.

while, until và mẫu đọc file

```
count=1
while [ $count -le 5 ]; do
    echo "$count"
    count=$(( count + 1 ))
done
```

```
num=1
until [ $num -ge 5 ]; do
    echo "$num"
    num=$(( num + 1 ))
done
```

```
while IFS= read -r line; do
    echo "Dong: $line"
done < Hello.txt
```

Thành ngữ chuẩn IFS= read -r line

IFS= giữ khoảng trắng đầu/cuối dòng; -r không cho \ bị hiểu là escape. Không có hai thứ đó, dữ liệu bị biến dạng *âm thầm*. Thêm nữa: `cat f | while read ...` chạy trong **subshell** → biến gán bên trong *mất* sau vòng lặp.

case

```
case "$timeofday" in
    yes | y | Yes | YES )
        echo "Good Morning" ;;
    [nN]* )
        echo "Good Afternoon" ;;
    * )
        echo "Sorry, answer not recognized" >&2
        exit 1 ;;
esac
```

Ba điểm về case

1. case thực thi **mẫu khớp đầu tiên**, không phải mẫu khớp "tốt nhất" $\Rightarrow$ đặt mẫu cụ thể trước, mẫu tổng quát `*)` sau cùng.
2. Mỗi nhánh kết thúc bằng `;;` – quên là lỗi cú pháp.
3. Mẫu ở đây là **glob**, không phải regex: `[nN]*` = "bắt đầu bằng n hoặc N".

Hàm

```
sum() {
    echo $(( $1 + $2 ))      # "tra ve" ket qua bang cach IN ra stdout
}
result=$(sum 5 3)            # bat output bang the lenh
echo "Result: $result"       # -> Result: 8
```

Khi hàm được gọi, $1, $2, $#, $@ tạm thời là tham số *của hàm*; khi hàm kết thúc chúng được phục hồi. ($0 là ngoại lệ: vẫn là tên script.)

`return` trong shell KHÁC `return` trong C

Nó chỉ đặt *mã trả về* (0–255), và **0 nghĩa là thành công** – ngược với quy ước của C. Muốn trả *dữ liệu* thì `echo` rồi bắt bằng `$(...)`. Hệ quả: hàm không thể vừa in thông báo vừa "trả về" chuỗi qua stdout ⇒ đưa thông báo ra stderr.

Dùng `local`

Mặc định mọi biến trong shell là *toàn cục*: biến `i` trong hàm ghi đè biến `i` của vòng lặp gọi nó.

Nội dung

1. Mở đầu: vì sao bắt đầu bằng shell
2. Shell và cấu trúc dòng lệnh
3. Hệ thống tệp và đường dẫn
4. Quyền truy cập file
5. Lệnh quản lý file và thư mục
6. Chuyển hướng luồng (redirection)
7. Pipe, IPC và grep
8. Shell script
9. Gỡ lỗi và tra cứu
10. Bài tập thực hành
11. Tổng kết

Công cụ gỡ lỗi

Cách dùng	Tác dụng
`bash -x script.sh`	In từng lệnh *sau khi đã thế biến* – công cụ số một
`bash -n script.sh`	Chỉ kiểm tra cú pháp, không chạy
`set -x` / `set +x`	Bật/tắt trace cho *một đoạn* trong script
`set -e`	Thoát ngay khi có lệnh trả mã khác 0
`set -u`	Báo lỗi khi dùng biến chưa đặt (bắt lỗi gõ sai tên biến)
`set -o pipefail`	Pipeline trả mã lỗi nếu *bất kỳ* lệnh nào trong đó lỗi
`shellcheck script.sh`	Phân tích tĩnh; phát hiện đa số bẫy trong buổi hôm nay

```
#!/bin/bash
set -euo pipefail      # nen co o moi script viet moi
```

Quan trọng

`set -euo pipefail` biến lỗi *ngầm* thành lỗi *rõ ràng*. Không có nó, script sai vẫn "chạy xong" và trả về 0, làm tưởng đúng mà lại sai.

Đọc output của bash -x

```
$ bash -x fib.sh
+ n=5
+ a=0
+ b=1
+ '[' 5 -ge 0 ']'
+ echo 0
0
```

Điểm mạnh nhất

Bạn thấy giá trị **sau khi** thế biến, nên phát hiện ngay biến rỗng hoặc bị tách từ. Ví dụ dòng `+ '[' " -eq 5 ']'` tố giác một biến rỗng không được bọc nháy.

Tối ưu

Ưu tiên built-in thay vì lệnh ngoài (mỗi lệnh ngoài là một `fork+exec`): `$((a+b))` nhanh hơn `$(expr $a + $b)` hàng chục lần trong vòng lặp.

Tra cứu: man và các mục của nó

Lệnh	Dùng khi
`man ls`	Tài liệu đầy đủ. `/tukhoa` tìm, `n` kết quả tiếp, `q` thoát
`man -k keyword`	Không biết tên lệnh, chỉ biết việc cần làm
`man 2 write`	Chỉ định *mục* (section)
`cmd -help`	Tóm tắt nhanh khi chỉ cần nhớ một tuỳ chọn
`help cd`	Dành cho *built-in* – built-in không có trang man riêng
`type -a name`	Biết `name` là built-in, alias, hàm hay lệnh ngoài

Các mục của man – rất quan trọng cho phần C

Mục **1** = lệnh người dùng, **2** = **system call**, **3** = hàm thư viện C, **5** = định dạng file, **7** = khái niệm tổng quát. Vì vậy `man write` cho bạn lệnh gửi tin nhắn, còn `man 2 write` mới là system call bạn cần. Tập gõ ngay từ tuần này: `man 2 open`, `man 2 fork`, `man 7 signal`.

Nội dung

1. Mở đầu: vì sao bắt đầu bằng shell
2. Shell và cấu trúc dòng lệnh
3. Hệ thống tệp và đường dẫn
4. Quyền truy cập file
5. Lệnh quản lý file và thư mục
6. Chuyển hướng luồng (redirection)
7. Pipe, IPC và grep
8. Shell script
9. Gỡ lỗi và tra cứu
10. Bài tập thực hành
11. Tổng kết

Kế hoạch thời gian và các điểm dừng

Thời lượng	Nội dung	Sinh viên làm
0–15’	Mục tiêu, shell, cấu trúc dòng lệnh	Mở terminal, `whoami`, `pwd`, `man ls`
15–35’	Hệ thống tệp, quyền, lệnh quản lý file	**Bài tập 1, 2**
35–50’	Redirection, pipe, grep	**Bài tập 3, 4**
50–60’	Tiến trình, signal	Thử `ps`, `pgrep`, `kill`
60–80’	Shell script: biến, điều kiện, vòng lặp, hàm	**Bài tập 5, 6**
80–90’	Gỡ lỗi, tra cứu, giao bài về nhà	**Bài tập 7, 8**

Quy tắc của buổi thực hành

Gõ tay, không copy–paste. Sau mỗi lệnh, **kiểm chứng** bằng `ls -l`, `cat` hoặc `wc -l`. Phần lớn giá trị của Lab 1 nằm ở việc *đọc được thông báo lỗi* – nên hãy chủ động gõ sai một vài lần.

Bài tập 1 – Quản lý file và thư mục

1. Về thư mục người dùng
2. Trong đó, tạo hai thư mục con `Slides`, `Labs` và file `Solution.txt`
3. Vào thư mục `Labs`
4. Trong `Labs`, tạo thư mục con `Lab1` và file `Ex1.txt`
5. Copy `Ex1.txt` từ `Labs` sang `Lab1`
6. Xoá `Ex1.txt` trong `Labs`
7. Đổi tên `Ex1.txt` thành `Exercise1.txt`
8. Thêm quyền ghi cho others đối với `Exercise1.txt`
9. Xoá thư mục `Slides`

Gợi ý: hai bước 7 và 8 là chỗ dễ sai nhất

Sau bước 6, `Ex1.txt` còn tồn tại *ở đâu*? Hãy `ls -l` trước khi gõ bước 7.

Bài tập 1 – lời giải và kiểm chứng

```
cd ~                                          # hoac chi can: cd
mkdir Slides Labs                             # mkdir nhan NHIEU doi so cung luc
touch Solution.txt
cd Labs
mkdir Lab1 ; touch Ex1.txt
cp Ex1.txt Lab1/                              # dau / lam ro Lab1 la THU MUC dich
rm Ex1.txt                                    # ban trong Lab1 con nguyen
mv Lab1/Ex1.txt Lab1/Exercise1.txt    # doi ten = di chuyen trong cung thu
    muc
chmod o+w Lab1/Exercise1.txt
ls -l Lab1/                                   # KIEM CHUNG: -rw-r--rw-  <- others co
    'w'
rmdir ~/Slides                                # chi xoa duoc neu Slides RONG
```

Ba nhận xét

(7) Nếu gõ `mv Ex1.txt Exercise1.txt` khi đang ở Labs → "No such file". **(8)** "others" là o, không phải a; `chmod 646` cho kết quả giống nhưng là *đặt lại* chứ không *thêm*. **(9)** `rmdir`

Bài tập 2 – ghi lại kết quả chuỗi lệnh

Vào Lab1, chạy lần lượt và **ghi lại kết quả**: `cd` → `pwd` → `ls -al` → `cd .` → `pwd` → `cd ..` → `pwd` → `ls -al` → `cd ..` → `pwd` → `ls -al`

#	Lệnh	Thư mục sau lệnh	Giải thích
1	`cd`	`/home/user`	**Không in gì**; `cd` ≡ `cd $HOME`
2	`pwd`	`/home/user`	In `/home/user`
3	`ls -al`	`/home/user`	Nội dung nhà, kể cả `.bashrc`, `.`, `..`
4	`cd .`	`/home/user`	**Không đổi gì** – lệnh hợp lệ nhưng vô tác dụng
5	`pwd`	`/home/user`	Bằng chứng cho bước 4
6–7	`cd .. ; pwd`	`/home`	Lên thư mục cha
8	`ls -al`	`/home`	Thư mục nhà của mọi người dùng
9–10	`cd .. ; pwd`	`/`	Lên tiếp: tới gốc
11	`ls -al`	`/`	`bin boot dev etc home lib proc tmp usr var`

Thử thêm bước 12: `cd ..` rồi `pwd`

Tại `/` thì `cd ..` *vẫn ở* `/` – gốc là cha của chính nó. Câu hỏi vấn đáp rất hay gặp.

Bài tập 3 – ghi 5 dòng vào Hello.txt

```
echo "Hello everyone!"                                          >  Hello.txt
echo "This is Advanced Programming Techniques course."          >> Hello.txt
echo "There are many exercises for you in this course."         >> Hello.txt
echo "Let's practice what we learn in this course with exercises." >> Hello.
    txt
echo "Are you ready?"                                           >> Hello.txt
cat Hello.txt ; wc -l Hello.txt       # KIEM CHUNG: phai la 5 dong
```

Dòng đầu >, các dòng sau »

Dùng > cho mọi dòng ⇒ file chỉ còn *dòng cuối*, mà `cat` vẫn hiện "có nội dung". Dùng » cho cả dòng đầu và chạy lại lần hai ⇒ nội dung nhân đôi. **Luôn `wc -l`.**

Bài học sống về quoting ở dòng 4

`Let's` nằm trong nháy kép nên dấu nháy đơn là ký tự thường – không sao. Nếu bọc bằng nháy đơn, shell hiểu nháy thứ hai là dấu *đóng* và treo chờ input (dấu nhắc >). Ctrl+C để thoát.

Bài tập 4 – I/O redirection và pipe

Dùng redirection, pipe và các lệnh Unix để:

1. Đếm số lần từ `course` xuất hiện trong `Hello.txt`, lưu vào `Ex4_results.txt`
2. In dòng thứ 3 của `Hello.txt`, đếm số từ của dòng đó, ghi thêm vào cuối file kết quả *(gợi ý: `sed`, `wc`, pipe)*
3. Lưu tên và thuộc tính của các file, thư mục trong thư mục người dùng
4. Đếm tổng số file thường và thư mục trong thư mục người dùng, ghi thêm vào cuối file kết quả
5. Đếm tổng số thư mục trong thư mục người dùng, ghi thêm vào cuối
6. Đếm tổng số tiến trình hiện có trong hệ thống, ghi thêm vào cuối

Chú ý ba chữ "ghi thêm vào cuối"

Từ câu 2 trở đi **phải** dùng ». Một dấu > là mất hết kết quả trước đó.

Bài tập 4 – lời giải

```
# 4.1 -- dem SO LAN xuat hien (khong phai so dong)
grep -o -w "course" Hello.txt | wc -l  >  Ex4_results.txt

# 4.2 -- dong thu 3, dem so tu  (-> 9 tu)
sed -n '3p' Hello.txt | wc -w          >> Ex4_results.txt

# 4.3 -- ten va thuoc tinh
ls -la ~                               >  Ex4_dir_listing.txt

# 4.4 -- so file thuong + thu muc
ls -l ~ | grep -c '^[-d]'              >> Ex4_results.txt

# 4.5 -- so thu muc
ls -l ~ | grep -c '^d'                 >> Ex4_results.txt

# 4.6 -- so tien trinh
ps -e --no-headers | wc -l             >> Ex4_results.txt
```

Bài tập 4 – ba điểm cần giải thích được

4.1 – vì sao không dùng `grep -c`?

`-c` đếm *số dòng chứa*. Với file Bài 3, mỗi dòng có nhiều nhất một "course" nên hai cách *tình cờ* cùng cho 3 – sự trùng khớp này che mất lỗi khái niệm. Tự chứng minh:

```
echo "course course course" >> Hello.txt
grep -c "course" Hello.txt          # -> 4  (4 DONG co chua)
grep -o "course" Hello.txt | wc -l  # -> 6  (6 LAN xuat hien)  <- dung
```

4.4 – hai cách cho hai con số khác nhau

Thư mục thử nghiệm: 2 thư mục thường, 1 thư mục ẩn, 4 file thường, 1 file ẩn, 1 symlink. `ls -l | grep -c '^[-d]'` → **7** (bỏ file ẩn; symlink có ký tự `l`). `find ~ -maxdepth 1 -mindepth 1 ...` → **9** (tính cả file ẩn). *Cái sai là báo cáo một con số mà không nói rõ định nghĩa.*

4.6 – `ps -e | wc -l` lệch 1 đơn vị

Vì dòng tiêu đề `PID TTY TIME CMD` cũng bị đếm: lỗi off-by-one ở cấp shell.

Bài tập 5 – script `greeting.sh`

Viết script `greeting.sh` thực hiện:

a) Chứa chú thích với tên bạn, tên script và mục đích của script

b) Hiển thị chuỗi "Hello + username"

c) Hiển thị tên và phiên bản của hệ điều hành

d) Hiển thị group ID (GID) của người dùng hiện tại

e) Hiển thị ngày và giờ hiện tại

h) Hiển thị danh sách tất cả file trong thư mục người dùng

i) Hiển thị giá trị các biến `TERM`, `PATH`, `HOME`

j) Hiển thị "Goodbye + giờ hiện tại"

Yêu cầu (a) bị bỏ qua nhiều nhất – và nó được chấm điểm

Khối chú thích đầu file là quy ước nghề nghiệp: người đọc tiếp theo, kể cả chính bạn sáu tháng sau, cần biết script làm gì mà không phải đọc mã.

Bài tập 5 – lời giải

```
#!/bin/bash
# greeting.sh - <Ho ten> - <MSSV>
# Muc dich: in thong tin nguoi dung, he thong va bien moi truong.

echo "Hello ${USER:-$(whoami)}"
echo "Operating system : $(uname -s) $(uname -r)"
echo "GID              : $(id -g)  (group: $(id -gn))"
echo "Current date/time: $(date '+%Y-%m-%d %H:%M:%S')"
echo "--- Files in $HOME ---"
ls -la "$HOME"
echo "TERM = ${TERM:-<not set>}"
echo "PATH = $PATH"
echo "HOME = $HOME"
echo "Goodbye $(date '+%H:%M:%S')"
exit 0
```

Bốn lựa chọn cần giải thích được

`${USER:-$(whoami)}`: `$USER` có thể rỗng khi chạy từ cron – `uname -s -r` dễ đọc hơn `uname -a` – `id -g` in GID số, `id -gn` in tên nhóm – `"$HOME"` có nháy vì đường dẫn có thể chứa khoảng trắng.

Bài tập 6 – script `test.sh`

Hiển thị: tên chương trình, số tham số, tham số đầu tiên, tất cả tham số.

```
#!/bin/bash
# test.sh - In thong tin ve tham so dong lenh
echo "Program name         : $0"
echo "Number of parameters: $#"
echo "First parameter      : ${1:-<none>}"
echo "All parameters       : $@"
i=1
for arg in "$@"; do
    echo "  parameter[$i] = [$arg]"     # chung minh "$@" giu nguyen cau truc
    i=$(( i + 1 ))
done
exit 0
```

Đặt tên script là `test` (không có `.sh`) là một cái bẫy kinh điển

`test` là *built-in* (chính là `[`) ⇒ gõ `test` chạy built-in, không in gì, khiến bạn tưởng script "không chạy". **Trước khi đặt tên lệnh mới, chạy `type <ten>`.**

Bài tập 6 – kiểm chứng

```
./test.sh                          # -> Number of parameters: 0
./test.sh a b c                    # -> 3 tham so
./test.sh "Nguyen Van A" 2024      # -> 2 tham so
                                   #    parameter[1] = [Nguyen Van A]
```

Trường hợp thứ ba là phép thử thật sự

Nếu bài làm của bạn in `parameter[1] = [Nguyen]` và `parameter[2] = [Van]` thì bạn đã dùng `$@` hoặc `$*` *không có nháy kép*. Luôn kiểm chứng script bằng một tham số có khoảng trắng.

Bài tập 7 – chuyển thập phân sang thập lục phân

```
#!/bin/bash
# dec2hex.sh - Chuyen so nguyen thap phan sang thap luc phan
if [ $# -ne 1 ]; then
    echo "Usage: $0 <decimal_integer>" >&2 ; exit 1
fi
if ! [[ "$1" =~ ^-?[0-9]+$ ]]; then
    echo "Error: '$1' is not a decimal integer" >&2 ; exit 2
fi
hex=$(echo "obase=16; $1" | bc)
echo "$1 (dec) = $hex (hex)"
exit 0
```

Cơ chế

bc là máy tính tuỳ ý độ chính xác; biến obase đặt *cơ số đầu ra*. Ta đưa chuỗi "obase=16; 255" vào stdin của bc qua pipe → bc in FF; $(...) bắt kết quả vào biến.

Bài tập 7 – bốn cách giải và liên hệ với tuần 2

```
printf '%X\n' 255           # printf cua bash: nhanh nhat, khong goi lenh
    ngoai
echo "obase=16; 255" | bc   # bc: theo goi y cua de bai
echo $(( 16#FF ))           # NGUOC LAI: hex -> dec, cu phap 16#... cua bash
echo "ibase=16; FF" | bc    # ibase dat co so DAU VAO
```

Thứ tự obase / ibase

Khi dùng cả hai, phải đặt obase *trước* ibase, nếu không obase sẽ bị đọc theo cơ số vào mới.

Nối với bài giảng tuần 2

Hex là cách viết gọn cho nhị phân: 1 chữ số hex = 4 bit, 2 chữ số hex = 1 byte, 0xFF = 255. Bài tập 7 chính là phiên bản shell của phép chuyển đổi mà tuần 2 yêu cầu làm bằng C.

Bài tập 8 – dãy Fibonacci đến số thứ n

$F(0) = 0$, $F(1) = 1$, $F(n) = F(n-1) + F(n-2)$ với $n > 1$. Nhập n từ người dùng, in từ $F(0)$ đến $F(n)$.

```
#!/bin/bash
read -rp "Nhap n: " n
if ! [[ "$n" =~ ^[0-9]+$ ]]; then
    echo "Loi: n phai la so nguyen khong am" >&2 ; exit 1
fi
a=0 ; b=1                # a = F(i), b = F(i+1)
echo -n "Fibonacci: "
for (( i=0; i<=n; i++ )); do
    echo -n "$a "
    tmp=$(( a + b )) ; a=$b ; b=$tmp    # cua so truot
done
echo
```

Kiểm chứng: n=0 → 0; n=1 → 0 1; n=10 → 0 1 1 2 3 5 8 13 21 34 55.

Bài tập 8 – ba lỗi thường gặp

Đọc kỹ trước khi nộp

1. **Sai số phần tử.** Đề yêu cầu in *từ F(0) đến F(n)*, tức $n+1$ số. Vòng lặp `for ((i=0; i<n; i++))` in thiếu một số.
2. **Không xử lý $n=0$ và $n=1$.** Nhiều bài in thẳng `0 1` trước vòng lặp $\Rightarrow$ với $n=0$ kết quả sai. Cách viết ở slide trước xử lý đồng nhất mọi $n \geq 0$ mà không cần trường hợp riêng – *luôn ưu tiên thiết kế không có case đặc biệt*.
3. **Tràn số.** Số nguyên bash là 64 bit có dấu: $F(93)$ còn đúng, $F(94)$ bắt đầu tràn và cho kết quả âm. Thử `n=95` để tự thấy.

Liên hệ hai chiều

Kỹ thuật "cửa sổ trượt" hai biến là bước đầu của tư duy *lập trình động*: đệ quy thuần cho $O(2^n)$, cách này cho $O(n)$ – với $n=40$ là khác biệt giữa vài giây và vài chục phút. Còn hiện tượng tràn số là minh hoạ trực quan cho bài về miền giá trị kiểu dữ liệu ở tuần 2.

Nội dung

1. Mở đầu: vì sao bắt đầu bằng shell
2. Shell và cấu trúc dòng lệnh
3. Hệ thống tệp và đường dẫn
4. Quyền truy cập file
5. Lệnh quản lý file và thư mục
6. Chuyển hướng luồng (redirection)
7. Pipe, IPC và grep
8. Shell script
9. Gỡ lỗi và tra cứu
10. Bài tập thực hành
11. Tổng kết

Câu hỏi kiểm tra trong báo cáo

1. Vì sao `cd` phải là built-in?
2. `./script.sh` khác `source script.sh` ở đâu?
3. `cmd > f 2>&1` khác `cmd 2>&1 > f` thế nào?
4. Vì sao `rm "$f"` an toàn hơn `rm $f`?
5. Tôi xoá được file mà tôi không có quyền `w` trên đó không?
6. `grep -c` khác `grep -o | wc -l` thế nào?
7. `a | b | c` chạy tuần tự hay đồng thời?
8. Mã trả về của `a | b` là của lệnh nào?
9. `kill -9` khác `kill -15`, nên dùng cái nào trước?
10. `"$@"` khác `"$*"` khi tham số có khoảng trắng?
11. `$((...))`, `$(...)`, `${...}` dùng làm gì?
12. `return 0` trong hàm shell: "trả về số 0" hay "thành công"?

Tiêu chí chấm bài

Tính đúng đắn – Tính tường minh (shebang, chú thích, tên biến) – Tính bền vững (bọc nháy, kiểm tra tham số, báo lỗi ra stderr, mã trả về) – Sự hiểu biết (giải thích được lựa chọn và nêu giả định) – Kiểm chứng.

Chuẩn bị cho tuần sau

Đọc trước

- K&R, *The C Programming Language*, Chương 1 & 2 (tr. 9–52)
- Michael Kerrisk, *The Linux Programming Interface*, Chương 7 (tr. 139–152)

Luyện thêm nếu còn thời gian

- Chạy `shellcheck` trên cả 4 script đã viết và sửa hết cảnh báo
- Thêm `set -euo pipefail` vào từng script rồi xem script nào vỡ – và vì sao
- Gõ `man 2 open`, `man 2 fork`, `man 7 signal` để làm quen với mục 2 và 7
- Viết lại Bài 8 bằng đệ quy rồi so thời gian với bản vòng lặp tại $n = 30$ (dùng `time`)

Tài liệu đọc thêm

William Shotts, *The Linux Command Line* (có bản miễn phí) – K.C. Wang, *Systems Programming in Unix/Linux*

Cảm ơn đã lắng nghe!